

Base Analysis Core Guide

Candidate R1 - Semantic and Syntax Core

Base semantica e sintattica della BA, volutamente senza worked examples e senza question pack

Questa guida isola il core semantico e sintattico della BA: definizioni, firme, roleKey, cardinalita', condition language, strutture locali, provenance, projection, candidate construct e open register. Worked examples e question-driven discrimination sono intenzionalmente esclusi e sono forniti dalla Complete Guide companion nello stesso checkpoint R42.

Stato	CANDIDATE / NON-NORMATIVE / NOT PROMOTION ELIGIBLE
Scopo di questa revisione	Completezza semantica e sintattica del core BA, separata dai layer pedagogici e interrogativi.
Authority BA corrente	DDTA_BASE_ANALYSIS_OPERATIONAL_GUIDE_R3.tex + BA0 R1 / BA1 R1 / BA2 R3 / BA3 R1 / BA4 R1 / BA5 R1.
Operator basis corrente	14 top-level operators. Nessun quindicesimo operatore e' ammesso.
Repository baseline di partenza	bea1e6567d3f861294378b201648b0853b3c8a43
Layer esempi	Intenzionalmente assente; disponibile nella Complete Guide companion di R42.
Data	11 settembre 2026

Promotion stop esplicito

Questa Core Guide non e' un successore autonomo della R3: e' una vista intenzionalmente ridotta al contratto semantico/sintattico. La Complete Guide companion conserva questo core e aggiunge worked examples e question-driven discrimination.

Indice

1	Scopo, lineage e regola di costruzione	6
1.1	Legenda epistemica	6
2	Che cos'è la Base Analysis	6
3	Contratto BA0–BA5 e gate BA6	7
4	Procedura operativa end-to-end	7
5	BAReferent	8
5.1	Quando creare un BAREferent	8
5.2	Quando NON creare un BAREferent	8
6	BAProposition	8
6.1	Invarianti	9
7	Criterio di dettaglio, minimalità e stopping rule	9
8	Operator selection procedure	9
9	RoleKey: significato e disciplina	10
10	Matrice normativa delle cardinalità	11
11	Operator: transfer	11
11.1	Definizione	11
11.2	Firma canonica	11
11.3	Campi	12
11.4	Invarianti e confini	12
11.5	Forbidden inference	12
12	Operator: produce	12
12.1	Definizione	12
12.2	Firma canonica	12
12.3	Campi	13
12.4	Invarianti e confini	13
12.5	Forbidden inference	13
13	Operator: create	13

13.1	Definizione	13
13.2	Firma canonica	13
13.3	Campi	13
13.4	Invarianti e confini	14
13.5	Forbidden inference	14
14	Operator: observe	14
14.1	Definizione	14
14.2	Firma canonica	14
14.3	Campi	14
14.4	Invarianti e confini	14
14.5	Open delta	15
15	Operator: transition	15
15.1	Definizione	15
15.2	Firma canonica	15
15.3	Campi	15
15.4	Invarianti e confini	15
15.5	Open delta	15
16	Operator: correlate	16
16.1	Definizione	16
16.2	Firma canonica	16
16.3	Campi	16
16.4	Invarianti e confini	16
17	Operator: reference	16
17.1	Definizione	16
17.2	Firma canonica	16
17.3	Campi	17
17.4	Invarianti e confini	17
18	Operator: dependOn	17
18.1	Definizione	17
18.2	Firma canonica	17
18.3	Campi	17
18.4	Invarianti e confini	17
19	Operator: consumeService	18

19.1	Definizione	18
19.2	Firma canonica	18
19.3	Campi	18
19.4	Invarianti e confini	18
20	Operator: realize	18
20.1	Definizione	18
20.2	Firma canonica	18
20.3	Campi	19
20.4	Invarianti e confini	19
21	Operator: assignResponsibility	19
21.1	Definizione	19
21.2	Firma canonica	19
21.3	Campi	19
21.4	Invarianti e confini	19
22	Operator: constrain	20
22.1	Definizione	20
22.2	Firma canonica	20
22.3	Campi	20
22.4	Invarianti e confini	20
23	Operator: classify	20
23.1	Definizione	20
23.2	Firma canonica	20
23.3	Campi	21
23.4	Invarianti e confini	21
24	Operator: decisionRule	21
24.1	Definizione	21
24.2	Firma canonica	21
24.3	Campi top-level	21
24.4	Invarianti e confini	21
25	Condition language di decisionRule	22
25.1	comparison	22
25.2	satisfies	22
25.3	allOf	22

25.4	anyOf	23
25.5	not	23
25.6	resultAssignment	23
26	Struttura locale candidate: selection	23
26.1	Working structure	23
26.2	Distinzioni obbligatorie	24
27	Pressure aperta: pipeline composition e order	24
27.1	Domande da porre	24
28	Pressure e strutture trasversali ancora aperte	25
29	Candidate construct R25 non ammessi	25
29.1	CC-01 consumeData	25
29.2	CC-02 provideService	25
29.3	CC-03 storedIn	26
29.4	CC-04 initiate	26
29.4.1	Working minimum signature	26
29.4.2	Semantica minima	26
29.4.3	Non implica	26
30	Working hypothesis respinte	26
31	BA3 - provenance, origin state e revalidation	27
31.1	Origin states	27
31.2	Tre responsabilita' distinte	27
31.3	Derived semantics	27
31.4	Change e revalidation	28
32	Diagnostics e unresolved semantics	28
32.1	Quando preferire una diagnosis a una proposition	28
33	BA5 - authoring canonico	28
34	BA4 - projection boundary	29
34.1	Coverage modes	29
34.2	Projection non autorizza inference upstream	29
35	Regression e acceptance della Base Analysis	29
35.1	Checklist pre-acceptance	30

36 Open register R25 da preservare nella prossima revisione	30
37 Cosa questa revisione non chiude	31
38 Gate per la prossima revisione con esempi	31
A Quick syntax sheet - top-level operators	32
B Quick syntax sheet - condition language	33
C Quick status sheet - non-current constructs	33

1 Scopo, lineage e regola di costruzione

Questa guida separa deliberatamente due obiettivi che nelle revisioni precedenti erano intrecciati:

1. **completezza semantica e sintattica:** ogni costruito deve avere significato, firma, ruoli, cardinalita', confini e stato;
2. **completezza pedagogica:** ogni costruito deve essere accompagnato da esempi source-grounded e controesempi.

La R2 chiude soltanto il primo obiettivo. Il secondo e' esplicitamente rinviato alla revisione successiva.

Regola cumulativa

Una candidate successore non puo' perdere una distinzione stabile della R3 per essere piu' breve. Ogni contenuto R3 viene quindi classificato come preservato, espanso, qualificato oppure esplicitamente rinviato al layer di esempi. Nessuna evidence R25 diventa authority soltanto perche' e' inclusa in questa guida.

1.1 Legenda epistemica

Etichetta	Significato
CURRENT / PRESERVED	Regola gia' authority corrente in R3/BA0-BA5.
R25 EVIDENCE	Evidenza positiva o pressure confermata dal secondo ciclo DermaTriage.
CANDIDATE / NOT ADMITTED	Costrutto sopravvissuto a test parziali ma non formalmente ammesso.
REJECTED	Working hypothesis respinta nello scope corrente.
OPEN / COUNTEREXAMPLE-GATED	Questione semanticamente reale ancora non chiusa.
CORE VIEW / NO EXAMPLES	La semantica e' descritta; esempi e question pack sono intenzionalmente esclusi da questa vista core.

2 Che cos'e' la Base Analysis

CURRENT / PRESERVED

Per una baseline documentale governata, la *Base Analysis* e' la rappresentazione analitica condivisa e methodology-neutral del significato di progetto necessario ai consumer downstream DDTA.

```
governed project documentation
  = project authority

accepted Base Analysis
  = shared analytical baseline

projection / threat analysis / tests
  = downstream consumers
```

La BA puo' normalizzare, rendere riusabile e rendere interrogabile un significato gia' governato. Non puo' completare silenziosamente la documentazione, inventare un commitment, scegliere una interpretazione non supportata o trasformare una domanda downstream in project truth.

Authority direction

Documentazione governata → BA → projection/analysis. Il feedback puo' risalire come richiesta di review, ma una modifica di project truth avviene soltanto tramite una nuova decisione documentale governata e successiva revalidation della BA.

3 Contratto BA0–BA5 e gate BA6**CURRENT / PRESERVED**

Layer	Responsabilita'
BA0	authority boundary, minimum sufficient analysis, unresolved state, feedback direction e anti-inference.
BA1	due sole famiglie first-class: <i>BAReferent</i> e <i>BAProposition</i> .
BA2 R3	proposition semantics, top-level operators, operator-scoped roleKey, cardinalita' e strutture locali.
BA3	provenance, origin state, derivation, review/freshness, continuity e revalidation.
BA4	projection, traceability, interpretation e coverage downstream.
BA5	canonical semantic registries, domain-scoped resolution e controlled authoring.
BA6	integrated validation gate. Non viene dichiarato chiuso da questa guida.

BA2 authority corrente

Le firme di operatori, roleKey e cardinalita' normative derivano da BA2_RELATION_ACTION_VOCABULARY_R3. Snapshot storici incorporati in altri contratti non sostituiscono BA2 R3.

4 Procedura operativa end-to-end**CURRENT / PRESERVED**

1. Pin di baseline, authority e source revision.
2. Lettura source-first del governed source set.
3. Identificazione del minimum shared meaning necessario.
4. Identificazione dei significati che richiedono identita' riusabile.
5. Creazione dei BAReferent necessari.
6. Formulazione delle BAProposition minime usando BA2 R3.
7. Verifica di firma, roleKey, cardinalita' e scope delle proposition.
8. Collegamento BA3 a source evidence, origin state e revalidation context.
9. Registrazione esplicita di ambiguity, conflict, diagnosis e NOT SPECIFIED.
10. Minimum-BA acceptance per lo scope dichiarato.

11. Regression completa verso le fonti e forbidden-inference review.
12. Method-pressure review soltanto su counterexample concreti.
13. Materializzazione della accepted case BA.
14. Projection/downstream consumption secondo BA4.
15. Governed change e BA3 revalidation quando necessario.
16. Integrated BA6 verdict soltanto dopo l'intero protocollo previsto.

5 BAREferent

CURRENT / PRESERVED

Un *BAREferent* e' un significato di progetto methodology-neutral che riceve identita' analitica indipendente quando tale identita' e' materialmente necessaria.

5.1 Quando creare un BAREferent

Creare un BAREferent quando il significato deve almeno una volta essere:

- riusato da piu' proposition;
- qualificato o vincolato;
- correlato con un altro significato;
- referenziato come target distinto;
- confrontato tra baseline;
- target di provenance/revalidation indipendente;
- oggetto di una domanda downstream materialmente distinta.

5.2 Quando NON creare un BAREferent

Non creare un referent soltanto perche' compare un sostantivo nel testo, perche' un tool desidera un nodo, perche' ogni verbo dovrebbe avere un oggetto o perche' una projection grafica risulterebbe piu' comoda.

Identity test

Se il significato scompare come identita' autonoma senza perdere alcuna distinzione governata, alcuna possibilita' di qualificazione o alcuna tracciabilita' necessaria, probabilmente non richiede un BAREferent separato.

6 BAProposition

CURRENT / PRESERVED

Una *BAProposition* e' una asserzione analitica formalizzata su uno o piu' BAREferent. La proposition applica un operatore e assegna BAREferent ai roleKey ammessi da quell'operatore.

BAProposition

operator -> canonical BA2 operator key
 roles -> operator-scoped role assignments
 local -> only operator-local structures when admitted

6.1 Invarianti

- una proposition non e' il referent che describe;
- roleKey sono scoped all'operatore, non proprieta' universali del referent;
- un ruolo obbligatorio mancante non viene riempito per default;
- un ruolo opzionale si usa soltanto se il significato e' governato;
- una proposition non deve contenere semantica non ammessa dalla propria firma;
- piu' proposition possono essere necessarie per preservare distinzioni diverse contenute nella stessa clausola sorgente.

7 Criterio di dettaglio, minimalita' e stopping rule

CURRENT / PRESERVED

more detail

only if needed to:
 preserve a governed distinction
 answer a material downstream question
 expose a missing governed fact

Stopping test

Abbiamo abbastanza semantica per formulare la domanda rilevante e distinguere evidence supportata, contraddetta o NOT SPECIFIED? Se SI, fermarsi. Se NO, identificare la smallest missing distinction o produrre una diagnosis. Non inventare il dettaglio mancante.

8 Operator selection procedure

CURRENT / PRESERVED

La scelta dell'operatore avviene dopo la ricostruzione del significato, non prima.

1. isolare la smallest governed semantic fact;
2. identificare i BAREferent necessari;
3. formulare le candidate interpretations materialmente differenti;
4. provare prima gli operatori esistenti;
5. verificare firma e roleKey senza adattamenti opportunistici;
6. applicare delete test e merge-first test;

7. verificare i confini con gli operatori semanticamente vicini;
8. se nessun costrutto preserva il significato, registrare pressure/diagnosis invece di inventare un nuovo operatore locale;
9. un nuovo operatore richiede formal admission separata.

Verb-to-operator mapping

Il verbo naturale nella documentazione non determina automaticamente l'operatore BA. Parole come "acquire", "invoke", "store", "select", "execute", "trigger" e "expose" possono descrivere semantiche differenti a seconda del significato governato.

9 RoleKey: significato e disciplina

CURRENT / PRESERVED

I roleKey definiscono la funzione svolta da un BAReferent *all'interno di una specifica proposition*. Non classificano universalmente il referent.

roleKey	Significato operativo
behavior	identita' riusabile dello specifico transfer quando il comportamento di conveyance deve essere qualificato o target di altre proposition.
source	origine del content trasferito.
destination	destinatario del content trasferito.
content	significato oggetto della conveyance.
actor	soggetto/capability che compie l'azione quando la firma lo richiede o lo ammette.
result	risultato reso disponibile o risultato governato dell'operazione.
input	significato effettivamente usato dall'operazione; non ogni dato disponibile e' automaticamente input.
created	nuovo item/event project-semantic stabilito da create.
observed	significato esistente letto o ispezionato da observe.
subject	identita' la cui state/lifecycle identity e' interessata da transition, oppure subject locale di satisfies.
fromState	stato precedente governato, se espresso dalla fonte.
toState	stato risultante governato richiesto da transition.
correlatedItem	elemento che deve mantenere same-context binding.
correlationContext	identita' che definisce il contesto condiviso della correlation.
referencer	elemento che mantiene un riferimento direzionale verso altro significato.
referenced	target del riferimento.
dependent	significato che richiede uno o piu' prerequisiti.
prerequisite	significato necessario al dependent.
consumer	utilizzatore effettivo di un service/capability.
service	service/capability effettivamente consumato.
provider	provider del service, soltanto se governato; non e' implicito nel servizio.

roleKey	Significato operativo
abstract	significato astratto indipendentemente governato da materializzare.
realization	significato concreto che materializza l'abstract.
responsibleParty	parte cui viene assegnata responsibility/authority.
responsibilityScope	ambito/oggetto della responsibility assegnata.
responsibilityKind	tipo governato di responsibility/authority.
constraintTarget	significato cui si applica una restriction governata.
constraintValue	valore, dominio o struttura di restriction governata.
classifiedReferent	elemento cui viene assegnato un semantic kind.
semanticKind	categoria methodology-neutral governata.

10 Matrice normativa delle cardinalita'

CURRENT / PRESERVED

Operator	Required	Optional / local
transfer	source 1; destination 1..*; content 1..*	behavior 0..1
produce	actor 1; result 1..*	input 0..*
create	actor 1; created 1..*	none
observe	actor 1; observed 1..*	result 0..*
transition	subject 1; toState 1	actor 0..1; fromState 0..1
correlate	correlatedItem 1..*; correlationContext 1	none
reference	referencer 1; referenced 1..*	none
dependOn	dependent 1; prerequisite 1..*	none
consumeService	consumer 1..*; service 1	provider 0..1
realize	abstract 1; realization 1..*	none
assignResponsibility	responsibleParty 1..*; responsibilityScope 1; responsibilityKind 1	none
constrain	constraintTarget 1; constraintValue 1..*	none
classify	classifiedReferent 1; semanticKind 1..*	none
decisionRule	actor 1; input 1..*; result 1	operator-local rule

11 Operator: transfer

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

11.1 Definizione

transfer rappresenta la conveyance governata di uno o piu' content da una source a una o piu' destination. Non implica di per se' protocollo, successo, ownership, storage, processing o service consumption.

11.2 Firma canonica

```

transfer
  behavior    -> BAREferent [0..1]
  source      -> BAREferent [1]
  destination -> BAREferent [1..*]
  content     -> BAREferent [1..*]

```

11.3 Campi

Role	Card.	Semantica
behavior	0..1	identita' del comportamento di transfer, solo se deve essere riusato/qualificato.
source	1	origine governata della conveyance.
destination	1..*	uno o piu' destinatari governati.
content	1..*	contenuto il cui trasferimento e' governato.

11.4 Invarianti e confini

- `transfer != produce`: produrre un risultato non prova che esso venga trasferito a un destinatario.
- `transfer != observe`: leggere qualcosa non prova una conveyance source-to-destination.
- `transfer != consumeService`: utilizzare un service non coincide con trasferire content.
- `transfer != initiate`: avviare/invocare qualcosa non implica conveyance.
- `interaction mode`, `channel`, `retries`, `timeout`, `queueing`, `acknowledgement` e `failure semantics` si aggiungono solo se governati e rappresentabili nel livello corretto.

11.5 Forbidden inference

Non inferire protocollo, sincronicit , cifratura, affidabilit , successo, persistenza, ownership della source/destination o topologia intermedia.

12 Operator: produce

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

12.1 Definizione

`produce` rappresenta un actor/capability che rende disponibile un risultato governato. Il risultato puo' gia' avere identita' semantica senza essere "creato" nel senso di nuova occurrence/item project-semantic.

12.2 Firma canonica

```

produce
  actor  -> BAREferent [1]
  result -> BAREferent [1..*]
  input  -> BAREferent [0..*]

```

12.3 Campi

actor	1	soggetto/capability che produce il risultato.
result	1..*	output/result reso disponibile.
input	0..*	input governati effettivamente usati, non semplicemente disponibili.

12.4 Invarianti e confini

- produce != create: output availability non prova nuova identity/occurrence.
- produce != transfer: la disponibilita' di un result non identifica source/destination di conveyance.
- produce != decisionRule: un risultato prodotto non implica un mapping condizionale governato.
- produce != observe: un result di una query puo' essere prodotto, ma la lettura dell'esistente e il result prodotto sono semantiche separabili.

12.5 Forbidden inference

Non inferire algoritmo, score, ranking, persistenza, destinazione, nuova identity o condition logic non esplicitamente governati.

13 Operator: create

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

13.1 Definizione

create rappresenta l'istituzione di una nuova identita'/occurrence/item project-semantic che prima dell'azione non esisteva nel significato governato.

13.2 Firma canonica

```
create
  actor    -> BAREferent [1]
  created  -> BAREferent [1..*]
```

13.3 Campi

actor	1	soggetto/capability che stabilisce la nuova identity/occurrence.
created	1..*	item/event project-semantic creato.

13.4 Invarianti e confini

- `create != produce`: generare un output non dimostra nuova identity.
- `create != transition`: cambiare stato dello stesso subject non crea un nuovo subject.
- `create != storedIn`: persistenza/at-rest association non prova creazione.
- nomi tecnici come `factory`, `generator`, `constructor` o `insert` non sono sufficienti.

13.5 Forbidden inference

Non inferire ID generation, storage technology, lifecycle successivo, ownership o correlation con altri elementi se non governati.

14 Operator: observe

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

14.1 Definizione

`observe` rappresenta `read/query/inspection` esplicita di significato esistente senza asserire che l'osservazione crei o modifichi l'oggetto osservato.

14.2 Firma canonica

```
observe
  actor      -> BReferent [1]
  observed   -> BReferent [1..*]
  result     -> BReferent [0..*]
```

14.3 Campi

<code>actor</code>	1	soggetto/capability che legge o ispeziona.
<code>observed</code>	1..*	significato esistente oggetto dell'osservazione.
<code>result</code>	0..*	eventuale result distinto prodotto dall'osservazione; CMD-OP04 ne mantiene aperta la revisione.

14.4 Invarianti e confini

- `observe != input`: il fatto che un dato sia input non prova un'operazione di lettura semanticamente governata.
- `observe != transfer`: ricevere content non equivale automaticamente a osservarlo.
- `observe != selection`: query/retrieval puo' avvenire dopo una selection; non stabilisce la membership selezionata.
- `observe != transition`: inspection non implica state change.

14.5 Open delta

OPEN / COUNTEREXAMPLE-GATED CMD-OP04: la rimozione di `observe.result` resta deferita a integrated review; questa candidate preserva quindi la firma R3.

15 Operator: transition

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

15.1 Definizione

transition rappresenta un cambiamento governato di stato/lifecycle dello *stesso* subject project-semantic.

15.2 Firma canonica

```
transition
  subject    -> BAREferent [1]
  toState    -> BAREferent [1]
  actor      -> BAREferent [0..1]
  fromState  -> BAREferent [0..1]
```

15.3 Campi

subject	1	identita' che persiste attraverso il cambiamento.
toState	1	stato risultante governato.
actor	0..1	soggetto che causa/realizza il cambiamento, solo se governato.
fromState	0..1	stato precedente governato, se noto.

15.4 Invarianti e confini

- stessa identity prima/dopo e' requisito centrale;
- selection di una versione non e' automaticamente transition;
- qualification result non e' automaticamente state transition;
- modifica di una property non prova lifecycle/state identity;
- supporto al rollback non prova che una transition specifica sia governata o automatica.

15.5 Open delta

OPEN / COUNTEREXAMPLE-GATED CMD-OP05: la distinzione state identity vs typed value resta deferita; nessuna estensione della firma R3 viene ammessa qui.

16 Operator: correlate

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

16.1 Definizione

`correlate` preserva un binding di same-context tra uno o piu' elementi e una identita' di contesto condivisa.

16.2 Firma canonica

```
correlate
  correlatedItem      -> BReferent [1..*]
  correlationContext -> BReferent [1]
```

16.3 Campi

<code>correlatedItem</code>	1..*	elemento/i che devono restare nello stesso contesto.
<code>correlationContext</code>	1	identita' che definisce il contesto condiviso.

16.4 Invarianti e confini

- `correlate` != `reference`: il `reference` puo' essere direzionale senza same-context invariant.
- `correlate` != `dependOn`: same-context binding non implica prerequisite.
- `correlate` != `transfer`: binding non implica conveyance.
- non usare `correlate` come generic association fallback.

17 Operator: reference

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

17.1 Definizione

`reference` rappresenta un riferimento direzionale esplicito da un `referencer` a uno o piu' target identificati, quando non e' governata una semantica piu' forte.

17.2 Firma canonica

```
reference
  referencer -> BReferent [1]
  referenced -> BReferent [1..*]
```

17.3 Campi

referencer	1	elemento che mantiene il riferimento.
referenced	1..*	target del riferimento.

17.4 Invarianti e confini

- `reference != correlate`: non garantisce same-context binding.
- `reference != dependOn`: non garantisce prerequisite.
- `reference != assignResponsibility`: non assegna authority/responsibility.
- non usare `reference` come fallback per relazioni semanticamente non comprese.

18 Operator: dependOn

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

18.1 Definizione

`dependOn` rappresenta una relazione di prerequisite: il dependent richiede il prerequisite per il significato governato in scope.

18.2 Firma canonica

```
dependOn
  dependent    -> BReferent [1]
  prerequisite -> BReferent [1..*]
```

18.3 Campi

dependent	1	significato che necessita del prerequisite.
prerequisite	1..*	prerequisite governati richiesti.

18.4 Invarianti e confini

- `temporal order alone` non e' dependency;
- `precedes != dependOn`;
- `pipeline membership` non e' dependency;
- `dependOn != consumeService`: un prerequisite puo' non essere un service consumato;
- la transitive closure puo' essere projection-derived e non deve essere materializzata come truth senza necessita'.

19 Operator: consumeService

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

19.1 Definizione

consumeService rappresenta l'uso effettivo di un service/capability da parte di uno o piu' consumer, senza trasferire automaticamente ownership o provider responsibility.

19.2 Firma canonica

```
consumeService
  consumer -> BReferent [1..*]
  service  -> BReferent [1]
  provider -> BReferent [0..1]
```

19.3 Campi

consumer	1..*	utilizzatore/i effettivi del service.
service	1	service/capability consumato.
provider	0..1	provider esplicitamente governato; non implicito.

19.4 Invarianti e confini

- endpoint, componente, tecnologia, dataset o information item non sono automaticamente service;
- consumeService != transfer;
- consumeService != provideService candidate;
- consumption non implica ownership, administration o infrastructure responsibility;
- un consumer requirement non prova che il service garantisca la property richiesta.

20 Operator: realize

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

20.1 Definizione

realize rappresenta la materializzazione concreta di un significato astratto che esiste indipendentemente come commitment/capability/contract governato.

20.2 Firma canonica

```
realize
  abstract      -> BReferent [1]
  realization   -> BReferent [1..*]
```

20.3 Campi

abstract	1	significato astratto indipendentemente governato.
realization	1..*	elemento/i concreti che lo materializzano.

20.4 Invarianti e confini

- `realize != perform/execute`: realizzare una capability non equivale a rappresentare l'atto di eseguirla;
- `realize != classify`: non assegna semplicemente un kind;
- `realize != assignResponsibility`: materialization non equivale a responsibility placement;
- deve esistere un abstract meaning indipendentemente governato.

21 Operator: assignResponsibility

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

21.1 Definizione

`assignResponsibility` colloca una responsibility o authority governata su una o piu' responsible party per uno scope e un responsibility kind espliciti.

21.2 Firma canonica

```
assignResponsibility
  responsibleParty    -> BReferent [1..*]
  responsibilityScope -> BReferent [1]
  responsibilityKind  -> BReferent [1]
```

21.3 Campi

responsibleParty	1..*	parte/i cui e' attribuita la responsibility.
responsibilityScope	1	ambito governato della responsibility.
responsibilityKind	1	tipo di responsibility/authority governato.

21.4 Invarianti e confini

- `provider != responsible party` per default;
- `performer/actor != responsible party` per default;
- `producer/consumer != responsible party` per default;
- `realization != responsibility`;
- mera partecipazione non assegna authority.

22 Operator: constrain

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

22.1 Definizione

`constrain` rappresenta una restriction governata, riusabile e interrogabile applicata a un target identificato.

22.2 Firma canonica

```
constrain
  constraintTarget -> BAREferent [1]
  constraintValue  -> controlled value/structure [1..*]
```

22.3 Campi

<code>constraintTarget</code>	1	significato esatto su cui insiste la restriction.
<code>constraintValue</code>	1..*	valore, dominio o struttura di constraint governata.

22.4 Invarianti e confini

- `constrain != decisionCondition`: una condition locale non e' automaticamente reusable constraint;
- `constrain != decisionRule`: un dominio ammesso non definisce necessariamente come scegliere un risultato;
- `selection bound` puo' coesistere con `constrain` ma non assorbe `selected membership`;
- `NOT_GOVERNED` non viene trasformato in constraint negativo.

23 Operator: classify

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

23.1 Definizione

`classify` assegna a un referent uno o piu' semantic kind/category methodology-neutral governati.

23.2 Firma canonica

```
classify
  classifiedReferent -> BAREferent [1]
  semanticKind       -> BAREferent [1..*]
```

23.3 Campi

classifiedReferent	1	elemento cui viene assegnato il kind.
semanticKind	1..*	categoria/kind governati.

23.4 Invarianti e confini

- `classify != selection`: assegnare un kind non sceglie un membro tra candidati;
- `classify != transition`: cambiare category non prova lifecycle state change;
- `classify != generic property assignment`;
- il kind deve essere methodology-neutral e materialmente utile, non introdotto solo per il tool.

24 Operator: decisionRule

CURRENT / PRESERVED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

24.1 Definizione

`decisionRule` rappresenta un mapping governato da `input/conditions` a un risultato governato. E' appropriato quando il progetto governa non soltanto il dominio dei risultati, ma la regola che determina il risultato.

24.2 Firma canonica

```

decisionRule
  actor  -> BReferent [1]
  input  -> BReferent [1..*]
  result -> BReferent [1]

  rule
    IF      decisionCondition
    THEN    resultAssignment [1..*]
    ELSE    resultAssignment [0..*]
```

24.3 Campi top-level

actor	1	soggetto/capability che applica la regola.
input	1..*	input governati effettivamente usati dal mapping.
result	1	risultato governato dal rule mapping.
rule	1 local structure	struttura operator-local di condition e assignment.

24.4 Invarianti e confini

- `conditional action` non e' automaticamente `decisionRule`;

- non-sufficiency statement non e' automaticamente decisionRule;
- ranking/selection non e' automaticamente decisionRule;
- capability/support statement non e' automaticamente decisionRule;
- un allowed result domain da solo non definisce il mapping;
- property/value interni non vengono inventati per soddisfare la sintassi.

25 Condition language di decisionRule

CURRENT / PRESERVED

R25 EVIDENCE

Le condition form sono strutture locali. Non aumentano il conteggio dei top-level operator.

25.1 comparison

```
comparison
  referent      -> BReferent [1]
  property      -> controlled semantic key [1]
  comparisonKey -> equals | notEquals
  value         -> typed local value | BReferent [1]
```

referent	elemento la cui property viene confrontata.
property	semantic key controllata; resta obbligatoria nel contratto corrente.
comparisonKey	operazione di confronto ammessa. Current: equals, notEquals.
value	valore tipizzato o BReferent contro cui avviene il confronto.

CL-01 - ordered/scalar comparison vocabulary

Il ciclo R25 ha prodotto forte evidence per confronti ordinati/scalari che superano equals/notEquals. Le relazioni concettuali <, >, <=, >= sono pressure reale, ma il controlled vocabulary normativo e la relativa typing discipline non sono ancora chiusi. Non usarli come vocabulary BA2 current senza formal delta.

25.2 satisfies

CURRENT / PRESERVED

```
satisfies
  subject -> BReferent [1]
  condition -> BReferent [1]
```

Serve quando la fonte governa che un subject soddisfi una condition identificata, ma non governa un modello property/value interno che autorizzi una comparison.

25.3 allof

CURRENT / PRESERVED

R25 EVIDENCE

```
allOf
  childCondition -> decisionCondition [2..*]
```

Tutte le condizioni figlie devono valere. R25 fornisce forte evidence di conjunction semantics, senza trasformare allOf in top-level operator.

25.4 anyOf

CURRENT / PRESERVED

```
anyOf
  childCondition -> decisionCondition [2..*]
```

Almeno una condizione figlia deve valere. Il costrutto e' gia' corrente, ma R25 lo mantiene in stato PENDING: il positive control e' ancora insufficiente. Questo non equivale a rimozione dal contratto corrente.

25.5 not

CURRENT / PRESERVED

```
not
  childCondition -> decisionCondition [1]
```

Negazione locale di una condition. La frase "A non e' sufficiente per B" non equivale a not (B) e non deve essere compressa in una negazione logica falsa.

25.6 resultAssignment

CURRENT / PRESERVED

```
resultAssignment
  target -> BAREferent [1]
  value -> typed local value | BAREferent [1]
```

Rappresenta l'assegnazione del risultato governato nel ramo THEN/ELSE. Non autorizza property assignment generico fuori da decisionRule.

26 Struttura locale candidate: selection

CANDIDATE / NOT ADMITTED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

R25 ha dimostrato che bounded/ranked/recency selection contiene semantica non eliminabile in diversi controlli, senza giustificare un nuovo top-level operator.

26.1 Working structure

```
selection
  candidatePopulation -> OPEN TYPE [required]
```


selectionBasis	-> OPEN TYPE [required]
selectedResult	-> OPEN TYPE [required]
bound	-> OPEN TYPE [optional, only if governed]
orderingBasis	-> OPEN TYPE [optional, only if governed]

Typing non ancora congelato

Questa struttura e' REUSABLE_SELECTION_STRUCTURE / NON-NORMATIVE. R25 non decide ancora se i campi siano BAREferent, local structured values, condition-language structures o altro tipo gia' governato. criterion e selectionBasis restano una naming decision da chiudere nell'integrazione metodologica.

26.2 Distinzioni obbligatorie

ranking	!= selection
bound	!= selection
retrieval	!= selection
filtering	!= selection
best selection	!= full ranking

La membership selezionata deve restare distinguibile dal criterio, dall'eventuale ordinamento e dall'eventuale bound.

27 Pressure aperta: pipeline composition e order

OPEN / COUNTEREXAMPLE-GATED

R25 EVIDENCE

CORE VIEW / NO EXAMPLES

PR-02 PIPELINE_COMPOSITION_ORDER resta aperta. La guida deve quindi preservare le distinzioni senza introdurre prematuramente pipeline o precedes come top-level operator.

pipeline membership	
!= pipeline ordering	
!= precedes	
!= dependOn	
!= transfer	
!= initiate	

27.1 Domande da porre

- La fonte governa soltanto appartenenza a una stessa composizione?
- Governa un ordine totale o parziale?
- L'ordine e' semantico o soltanto descrittivo/implementativo?
- B richiede A come prerequisite, oppure A e' soltanto precedente?
- Esiste conveyance tra stadi oppure solo successione logica?
- La composizione richiede una reusable local structure oppure e' proiettabile da proposition gia' accettate?

Current disposition

Nuovo top-level operator: NOT JUSTIFIED. Reusable local composition structure: possibile, ma non ancora chiusa. Test successivo: membership vs ordering; precedes vs dependOn.

28 Pressure e strutture trasversali ancora aperte

ID	Topic	Stato / disciplina
PR-01	function/process/behavior identity / perform?	PRESSURE RETAINED; execution evidence esiste ma irreducibility non dimostrata.
PR-04	boundary/interaction association	OPEN; distinguere crossing, transfer, interface e initiation.
PR-07	structured information contract	OPEN; puo' appartenere a structure/-schema/composition anziche' dynamic relation.
PR-09	acquisition/refresh semantics	OPEN; esaurire operatori correnti prima di introdurre primitive.
PR-12	negative implication/non-sufficiency	OPEN; non comprimere in not.
PR-14	applicability/configuration binding	OPEN; provare qualifier/reusable structure prima di nuovo operatore.
OBS-OT-01	operation target / effect scope	LEVEL UNRESOLVED; verificare local structure/composition.

29 Candidate construct R25 non ammessi

29.1 CC-01 consumeData

REJECTED

Disposition: REJECT_REDUNDANT. Nei casi correnti il non-destructive data/evidence use e' rappresentabile senza una nuova primitive top-level; destructive-consume semantics non risultano governate.

29.2 CC-02 provideService

CANDIDATE / NOT ADMITTED

Positive provider/service evidence sopravvive l'exhaustion corrente, ma formal gate G1-G8, compatibility e final signature sono pendenti.

Signature non congelata

Questa guida non inventa roleKey o cardinalita' di provideService. Fino alla formal admission, il candidate resta evidence e non vocabulary BA2 corrente.

29.3 CC-03 storedIn

CANDIDATE / NOT ADMITTED

L'associazione at-rest sopravvive l'exhaustion corrente, ma formal admission, compatibility e final signature restano pendenti.

Signature non congelata

Questa guida non inventa un modello container/item, storage owner, persistence semantics o lifecycle. storedIn resta TESTED_POSITIVE / NOT_ADMITTED.

29.4 CC-04 initiate

CANDIDATE / NOT ADMITTED

29.4.1 Working minimum signature

```
initiate
  initiator -> BAREferent [1]
  initiated -> BAREferent [1]
```

29.4.2 Semantica minima

Rappresenta una relazione diretta in cui un referent avvia/richiede l'avvio di un altro significato governato. La signature non distingue ancora request vs start come kind separati.

29.4.3 Non implica

```
success
completion
transfer
service consumption
create
transition
automatic trigger
synchronous execution
```

Disposition: TESTED_POSITIVE / NOT_ADMITTED; formal G1-G8 e cross-corpus compatibility sono ancora richiesti.

30 Working hypothesis respinte

REJECTED

Hypothesis	Disposition corrente
generic consume	NOT JUSTIFIED.
generic modify	REJECTED - false semantics come generic operator.
generic persist	NOT JUSTIFIED AS GENERIC OPERATOR.

Hypothesis	Disposition corrente
generic expose	NOT JUSTIFIED - current corpus decomposable.
generic trigger	NOT JUSTIFIED AS SEPARATE PRIMITIVE; directed activation pressure routed to candidate initiate + condition.
verb-specific invoke	ABSORBED BY CC-04 initiate.
initiationKind	REJECT REDUNDANT / OVER-SPECIFIC nel current
REQUEST START	evidence set.

31 BA3 - provenance, origin state e revalidation

CURRENT / PRESERVED

Ogni BA element meaning-bearing deve risolvere a una baseline governata e alla provenance richiesta dal contratto BA3.

31.1 Origin states

GROUNDED
 DERIVED
 DIAGNOSTIC_UNRESOLVED

31.2 Tre responsabilita' distinte

sourceLink
 dove e' supportato il significato

 derivationBasisBinding
 quali elementi governati e quali ruoli fondano una derivazione

 revalidationContext
 quale change/review context determina freshness e revalidation

Provenance non deve simulare una relazione BA2 mancante.

31.3 Derived semantics

Un elemento DERIVED richiede almeno:

- derivation rule identificabile e versionata;
- basis role-bound;
- applicability contract;
- output semantic contract;
- lineage fino a governed source.

31.4 Change e revalidation

```
source change
  -> explicit impact / revalidation context
  -> impacted BA elements
  -> PENDING_REVIEW / STALE where applicable
  -> RETAIN / REPLACE / RETIRE
  -> accepted BA for new baseline
  -> rebuild projections
```

Una accepted BA storica non viene mutata silenziosamente come se le analisi precedenti avessero usato la nuova semantica.

32 Diagnostics e unresolved semantics

CURRENT / PRESERVED

Vocabolario operativo:

```
MULTIPLE MATERIAL BA CANDIDATES
BA REQUIRES UNSUPPORTED FACT
BA LOSES GOVERNED MATERIAL DISTINCTION
BA EXPOSES SOURCE CONFLICT
NOT SPECIFIED
```

NOT SPECIFIED

NOT SPECIFIED non significa false, absent o denied. Significa che il significato necessario non è determinato dalla baseline governata corrente.

32.1 Quando preferire una diagnosis a una proposition

Usare una diagnosis quando la source non supporta una scelta univoca tra interpretazioni materialmente differenti, quando la firma corrente richiederebbe un fatto non governato o quando il metodo perderebbe una distinzione reale.

33 BA5 - authoring canonico

CURRENT / PRESERVED

I semantic key devono essere canonici nel proprio semantic domain.

- project referent names;
- BA2 operator keys;
- operator-scoped role keys;
- semantic kind keys;
- controlled property keys;
- controlled value domains;

- local-structure keys quando formalmente ammessi.

Sinonimia, traduzione, naming similarity o LLM similarity non stabiliscono equivalenza normativa automaticamente.

Registry boundary

BA5 governa canonicalita' e resolution; non puo' ridefinire semantica, roleKey o cardinalita' di un operatore BA2. Per questi aspetti l'authority corrente resta BA2 R3.

34 BA4 - projection boundary

CURRENT / PRESERVED

```
governed documentation
  -> accepted BA
    -> projection
      -> method-owned interpretation
```

Ogni meaning-bearing projection item deve tracciare alla BA.

34.1 Coverage modes

```
EXHAUSTIVE_FOR_DECLARED_SCOPE
SELECTIVE
```

Una selective projection puo' omettere elementi senza dichiararli falsi o assenti. Una method-owned interpretation puo' aggiungere semantica del metodo downstream, ma deve restare trace-bound e non diventare BA/project authority.

34.2 Projection non autorizza inference upstream

Un DFD, un graph, una threat taxonomy o una vista architetturale non possono costringere la BA ad aggiungere endpoint, trust boundary, actor, data flow o service relationship non governati.

35 Regression e acceptance della Base Analysis

CURRENT / PRESERVED

Dopo una material BA change o contract refinement:

1. ricostruire o aggiornare la BA contro la source pin;
2. verificare ogni proposition materiale;
3. verificare diagnostics e forbidden inference;
4. controllare property/constraint scope;
5. controllare request/case/identity/evaluation bindings;
6. verificare signatures e cardinalita';

7. rieseguire usefulness/projection checks appropriati;
8. cercare counterexample e semantic loss;
9. riaprire soltanto il minimo owning layer se necessario.

35.1 Checklist pre-acceptance

1. authority e source revision sono pinned?
2. ogni referent ha identity solo quando necessaria?
3. ogni proposition usa operator/role canonici?
4. tutte le cardinalita' required sono source-grounded?
5. ruoli optional sono presenti solo se governati?
6. e' stato introdotto un fatto non governato?
7. un NOT SPECIFIED e' diventato default?
8. provenance e revalidation sono sufficienti?
9. local structure candidate e' stata scambiata per normative vocabulary?
10. una candidate R25 e' stata presentata come admitted?
11. una projection puo' consumare la BA senza reinterpretare prose per shared meaning gia' rappresentato?
12. esiste un counterexample che richiede targeted reopen?

36 Open register R25 da preservare nella prossima revisione

OPEN / COUNTEREXAMPLE-GATED

ID	Tema	Stato corrente
PR-01	perform / execution identity	PRESSURE RETAINED; irreducibility not demonstrated.
PR-02	pipeline composition/order	OPEN.
PR-03	interface/path/invoke	RECONCILED / CC-04.
PR-04	boundary/interaction	OPEN.
PR-05	ordered comparison	ROUTE CL-01.
PR-06	scalar/property comparison	ROUTE CL-01.
PR-07	structured information contract	OPEN.
PR-08	at-rest storage	RECONCILED / CC-03.
PR-09	acquisition/refresh	OPEN.
PR-10	conditional action trigger	RECONCILED / CC-04 + CONDITION.
PR-11	data/evidence consumption	RECONCILED.
PR-12	negative implication/non-sufficiency	OPEN.
PR-13	bounded/ranked/recency selection	REUSABLE SELECTION STRUCTURE / NON-NORMATIVE.
PR-14	applicability/configuration binding	OPEN.

ID	Tema	Stato corrente
CMD-OP04	remove observe.result	DEFERRED.
CMD-OP05	transition state/value refinement	DEFERRED.
OBS-OT-01	operation-target/effect-scope	LEVEL UNRESOLVED.

37 Cosa questa revisione non chiude

Questa candidate non chiude:

- BA6 integrated acceptance;
- formal admission di provideService, storedIn, initiate;
- ordered/scalar comparator vocabulary oltre equals/notEquals;
- typing finale della selection structure;
- pipeline membership/order representation;
- function/process/behavior identity pressure;
- boundary/interactions, structured information, acquisition/refresh, negative non-sufficiency, applicability binding;
- CMD-OP04, CMD-OP05 e OBS-OT-01.

38 Gate per la prossima revisione con esempi

CORE VIEW / NO EXAMPLES

La prossima revisione deve mantenere integralmente questa struttura e aggiungere, senza rimuovere semantica:

1. almeno un positive o boundary control per ciascun top-level operator;
2. Facial Access come regression case storico dove disponibile;
3. DermaTriage come secondo/control case dove disponibile;
4. source clause → semantic facts → BAREferent → BAProposition;
5. rejected alternative routing;
6. forbidden inference;
7. errori frequenti;
8. esempi specifici per condition language, selection e pipeline/order;
9. dichiarazione esplicita “no clean positive control” quando il corpus non ne offre uno.

Acceptance criterion della Complete Guide companion

La guida sarà candidata alla promotion soltanto quando un lettore potrà sia ricostruire il contratto BA completo dalla guida, sia imparare ad applicarlo correttamente senza dover consultare conversazioni o working findings non integrati.

A Quick syntax sheet - top-level operators

```

transfer
  behavior    -> BReferent [0..1]
  source      -> BReferent [1]
  destination -> BReferent [1..*]
  content     -> BReferent [1..*]

produce
  actor  -> BReferent [1]
  result -> BReferent [1..*]
  input  -> BReferent [0..*]

create
  actor    -> BReferent [1]
  created -> BReferent [1..*]

observe
  actor    -> BReferent [1]
  observed -> BReferent [1..*]
  result   -> BReferent [0..*]

transition
  subject  -> BReferent [1]
  toState  -> BReferent [1]
  actor    -> BReferent [0..1]
  fromState -> BReferent [0..1]

correlate
  correlatedItem      -> BReferent [1..*]
  correlationContext -> BReferent [1]

reference
  referencer -> BReferent [1]
  referenced -> BReferent [1..*]

dependOn
  dependent    -> BReferent [1]
  prerequisite -> BReferent [1..*]

consumeService
  consumer -> BReferent [1..*]
  service  -> BReferent [1]
  provider -> BReferent [0..1]

realize
  abstract    -> BReferent [1]
  realization -> BReferent [1..*]

assignResponsibility
  responsibleParty -> BReferent [1..*]
  responsibilityScope -> BReferent [1]
  responsibilityKind -> BReferent [1]

constrain
  constraintTarget -> BReferent [1]
  constraintValue  -> controlled value/structure [1..*]

```

```

classify
  classifiedReferent -> BReferent [1]
  semanticKind      -> BReferent [1..*]

decisionRule
  actor  -> BReferent [1]
  input  -> BReferent [1..*]
  result -> BReferent [1]
  rule   -> operator-local structure

```

B Quick syntax sheet - condition language

```

comparison
  referent      -> BReferent [1]
  property      -> controlled semantic key [1]
  comparisonKey -> equals | notEquals
  value         -> typed local value | BReferent [1]

satisfies
  subject -> BReferent [1]
  condition -> BReferent [1]

allof
  childCondition -> decisionCondition [2..*]

anyOf
  childCondition -> decisionCondition [2..*]

not
  childCondition -> decisionCondition [1]

resultAssignment
  target -> BReferent [1]
  value  -> typed local value | BReferent [1]

```

C Quick status sheet - non-current constructs

```

selection
  REUSABLE_SELECTION_STRUCTURE / NON-NORMATIVE

provideService
  TESTED_POSITIVE / NOT_ADMITTED
  signature = NOT FROZEN

storedIn
  TESTED_POSITIVE / NOT_ADMITTED
  signature = NOT FROZEN

initiate
  TESTED_POSITIVE / NOT_ADMITTED
  working minimum:

```

```
initiator -> BAREferent [1]  
initiated -> BAREferent [1]
```

pipeline/order

OPEN

no new top-level operator admitted